ADITYA UNIVERSITY

UNIT-V

ADITYA UNIVERSITY

SYLLABUS

Backtracking: General Method, 8-Queens Problem, Sum of Subsets problem, Graph Coloring, 0/1 Knapsack Problem.

Branch and Bound: The General Method, 0/1 Knapsack Problem, Travelling Salesperson problem.

Text Book:

Fundamentals of Data Structures in C++, Horowitz, Ellis; Sahni, Sartaj; Mehta, Dinesh, 2ndEdition Universities Press.

What is Backtracking?

- Suppose you have to make a series of *decisions,* among various *choices,* where
  - You don't have enough information to know what to choose
  - Each decision leads to a new set of choices
  - Some sequence of choices (possibly more than one) may be a solution to your problem

- Backtracking is a methodical way of trying out various sequences of decisions, until you find one that "works".

Contd ...

The solution is based on finding one or more vectors that maximize, minimize, or satisfy a criterion function P(x1, …. xn).

All possible solutions require a set of constraints, divided into two categories:

- **Explicit Constraint:** Explicit constraints are rules that restrict each xi to take on values only from a given set. Ex: xn= 0 or 1.
- **Implicit Constraint:** Implicit Constraints are rules that determine which of the tuple in the solutions space of I satisfy the criterion function.

- There are three types of problems in backtracking –
  1. Decision Problem – In this, we search for a decision(yes/no) solution.
  2. Optimization Problem – In this, we search for the best solution.
  3. Enumeration Problem – In this, we find all feasible solutions.

ADITYA UNIVERSITY

Basic Terms

- **Solution Space / State Space Tree:** All tuples that satisfy the explicit constraints define a possible solution space for a particular instance T of the problem.

Example:

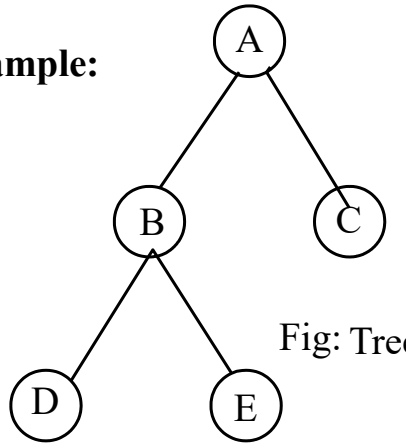


Fig: Tree organization of a solution space

Contd ...

- **Solution States:** These are the problem states S for which the path form the root to S defines a tuple in the solution space.

square nodes (□) indicate solution

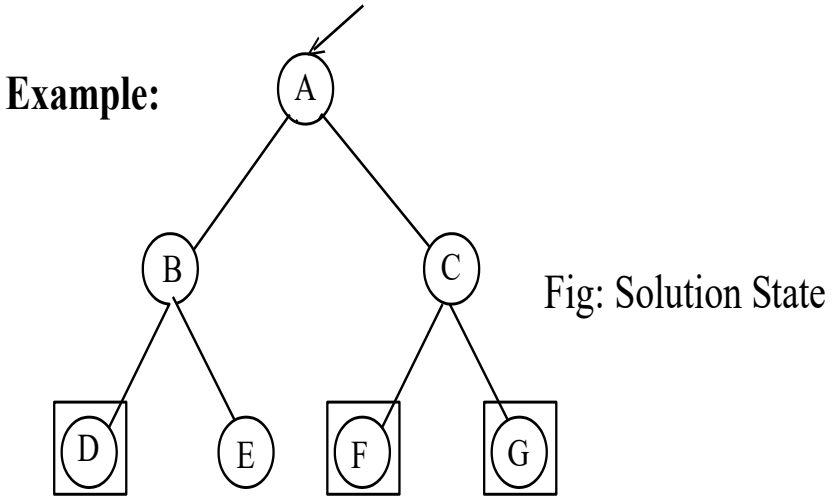


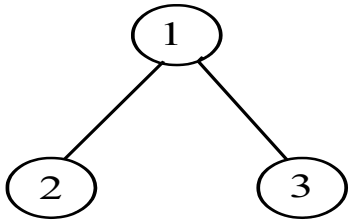
Fig: Solution State

- **Answer States:** These solution states S, for which the path from the root node to S defines a tuple that is a member of the set of solutions (i.e., it satisfies the implicit constraints) of the problem.

Eg: From above tree nodes D, F, and G are the Answer States.

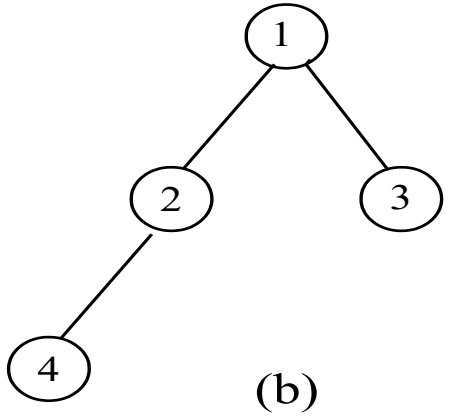
Contd ...

- **Live Node:** A node which has been generated but whose children have not yet been generated is live node.
- **E-Node:** The live nodes whose children are currently being generated is called the E-node (node being expanded).
- **Dead Node:** It is generated node, that is either not to be expanded further or one for which all of its children has been generated.



(a)

Nodes 1,2,and 3



(b)

Fig: Dead nodes

Nodes 1,3,and 4

ADITYA UNIVERSITY

Backtracking General Method

```
1   Algorithm Backtrack(k)
2   // This schema describes the backtracking process using
3   // recursion. On entering, the first k − 1 values
4   // x[1], x[2], . . . , x[k − 1] of the solution vector
5   // x[1 : n] have been assigned. x[ ] and n are global.
6   {
7       for (each x[k] ∈ T(x[1], . . . , x[k − 1])) do     //Test all possible solutions
8       {
9           if (B_k(x[1], x[2], . . . , x[k]) ≠ 0) then    //predicates the x[k] value
10          {
11              if (x[1], x[2], . . . , x[k] is a path to an answer node)
12                  then write (x[1 : k]);
13              if (k < n) then Backtrack(k + 1);
14          }
15      }
16  }
```

Fig: Recursive Backtracking

Contd ...

```
1    Algorithm IBacktrack(n)
2    // This schema describes the backtracking process.
3    // All solutions are generated in x[1 : n] and printed
4    // as soon as they are determined.
5    {
6         k := 1;
7         while (k ≠ 0) do
8         {
9              if (there remains an untried x[k] ∈ T(x[1], x[2], . . .,
10                  x[k − 1])  and B_k(x[1], . . . , x[k]) is true) then
11             {
12                  if (x[1], . . . , x[k] is a path to an answer node)
13                       then write (x[1 : k]);
14                  k := k + 1; // Consider the next set.
15             }
16             else k := k − 1; // Backtrack to the previous set.
17        }
18   }
```

Fig: Iterative Backtracking

ADITYA UNIVERSITY

1. N-queen problem

1) Start in the leftmost column

2) If all queens are placed return true

3) Try all rows in the current column.

Do following for every tried row.

a) If the queen can be placed safely in this row then mark this [row, column] as part of the solution and recursively check if placing queen here leads to a solution.

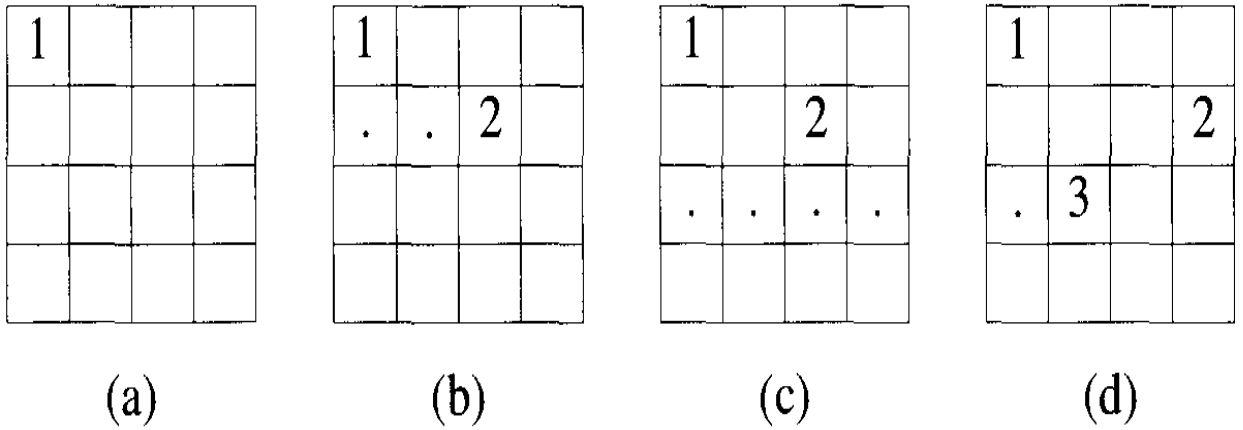
b) If placing queen in [row, column] leads to a solution then return true.

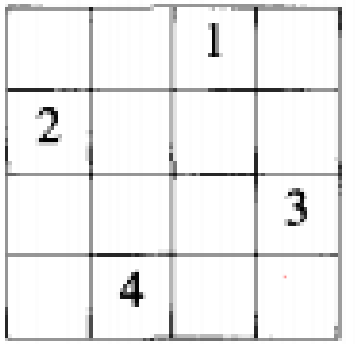
c) If placing queen doesn't lead to a solution then unmark this [row, column] (Backtrack) and go to step (a) to try other rows.

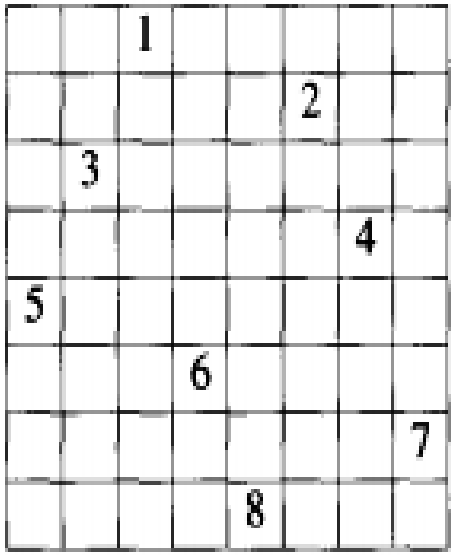
4) If all rows have been tried and nothing worked, return false to trigger backtracking.

ADITYA UNIVERSITY

n-queen problem

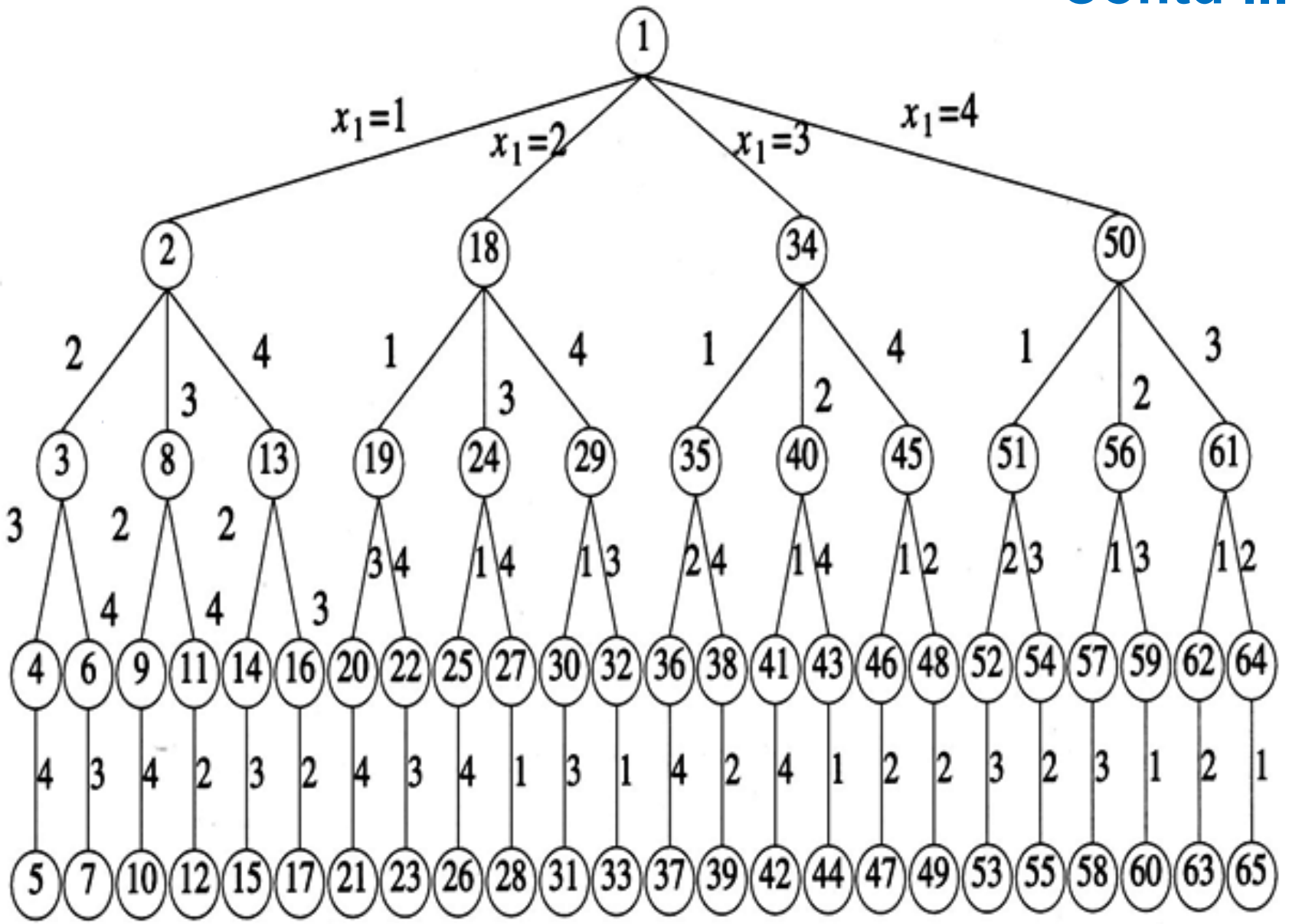






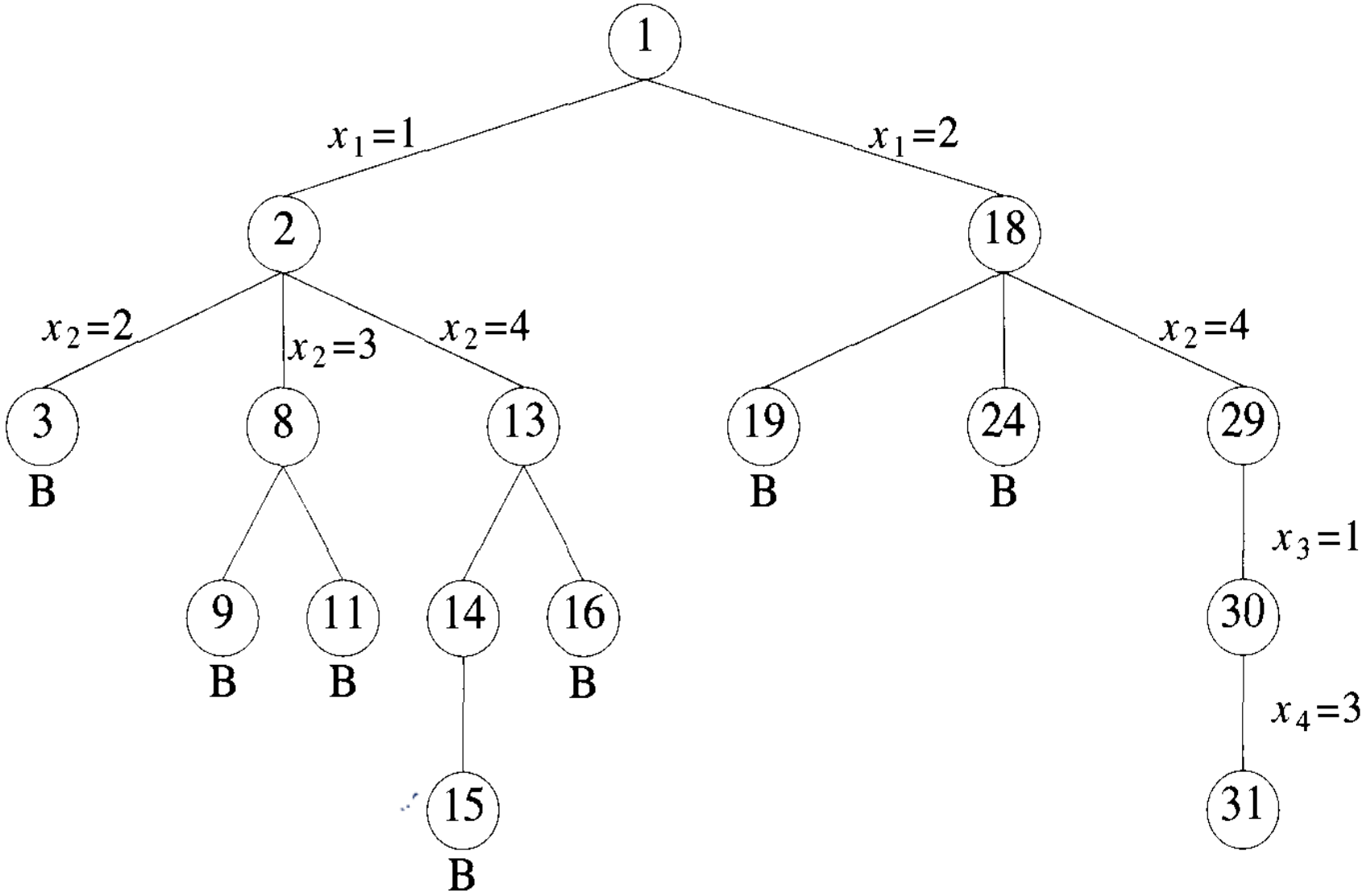
ADITYA UNIVERSITY

Contd ...



ADITYA UNIVERSITY

4-queen problem tree portion



ADITYA UNIVERSITY

Algorithm

```
1   Algorithm Place(k, i)
2   // Returns true if a queen can be placed in kth row and
3   // ith column. Otherwise it returns false. x[ ] is a
4   // global array whose first (k – 1) values have been set.
5   // Abs(r) returns the absolute value of r.
6   {
7       for j := 1 to k – 1 do
8           if ((x[j] = i) // Two in the same column
9               or (Abs(x[j] – i) = Abs(j – k)))
10                  // or in the same diagonal
11              then return false;
12      return true;
13  }
```

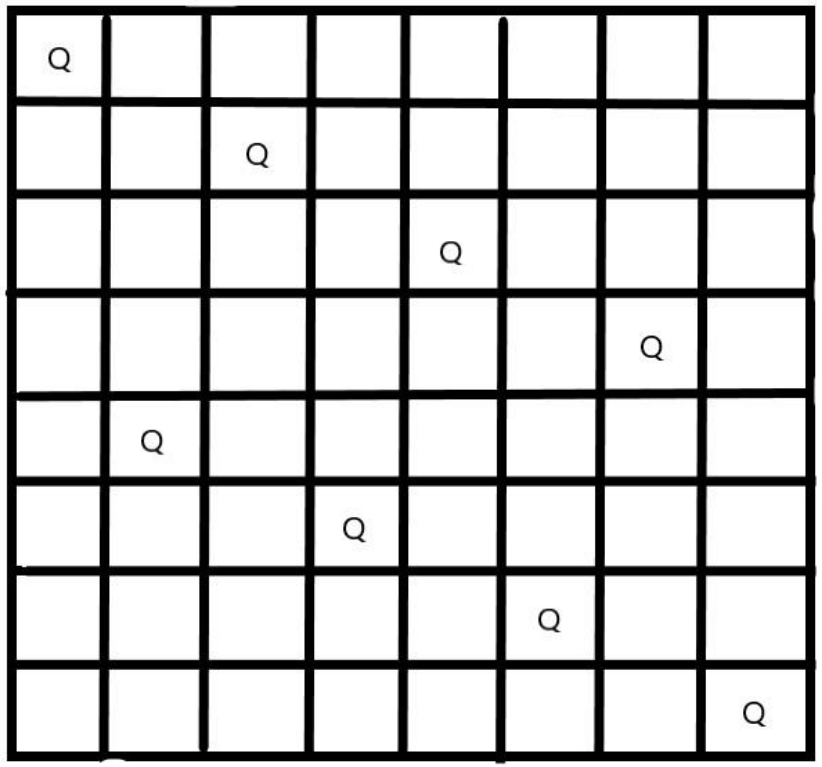
Contd …

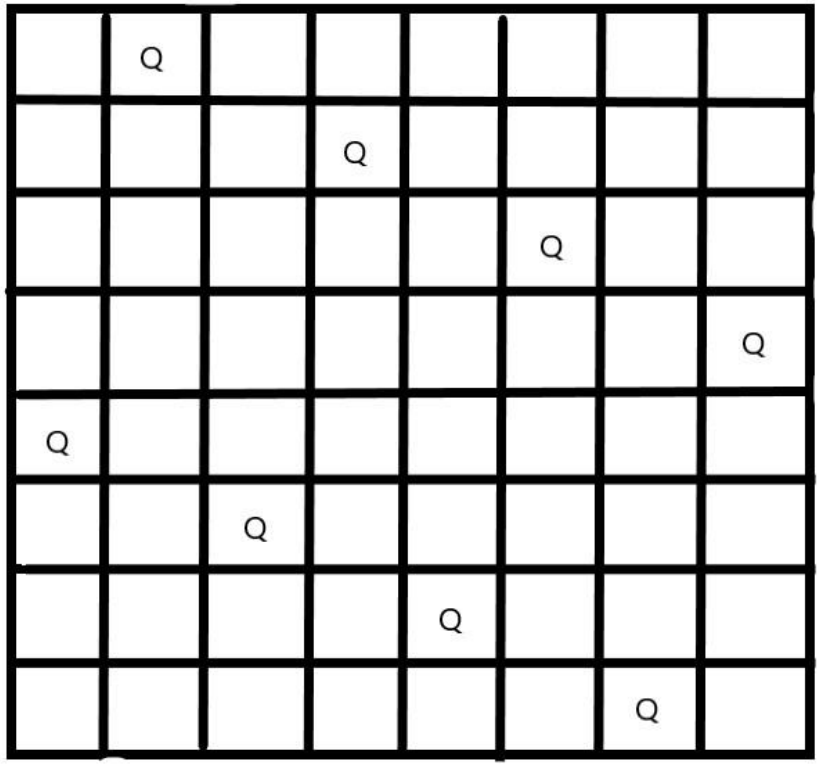
```
1    Algorithm NQueens(k, n)
2    // Using backtracking, this procedure prints all
3    // possible placements of n queens on an n × n
4    // chessboard so that they are nonattacking.
5    {
6        for i := 1 to n do
7        {
8            if Place(k, i) then
9            {
10               x[k] := i;
11               if (k = n) then write (x[1 : n]);
12               else NQueens(k + 1, n);
13           }
14       }
15   }
```

Time Complexity is O(n!) not the n^n

ADITYA UNIVERSITY

8-Queens





ADITYA UNIVERSITY

2. Sum of subsets

Subset sum problem is to find subset of elements that are selected from a given set whose sum adds up to a given number ‘M’. We are considering the set contains non-negative values. It is assumed that the input set is unique. Data can be showed as

$$(\sum Wi*Xi , K, \sum Wi)$$

Where Wi, is the corresponding element weight.

Xi, is the corresponding element solution vector value.

K is the which element is verifying now.

- If $\sum$ **(i is 1 to k)** Wi*Xi <= m, then only expands further, otherwise kill that possibility.
- If $\sum$ **(i is 1 to k)** Wi*Xi + $\sum$ **(i is k+1 to n)** Wi > m then only expands further, otherwise kill that possibility.

ADITYA UNIVERSITY

Algorithm

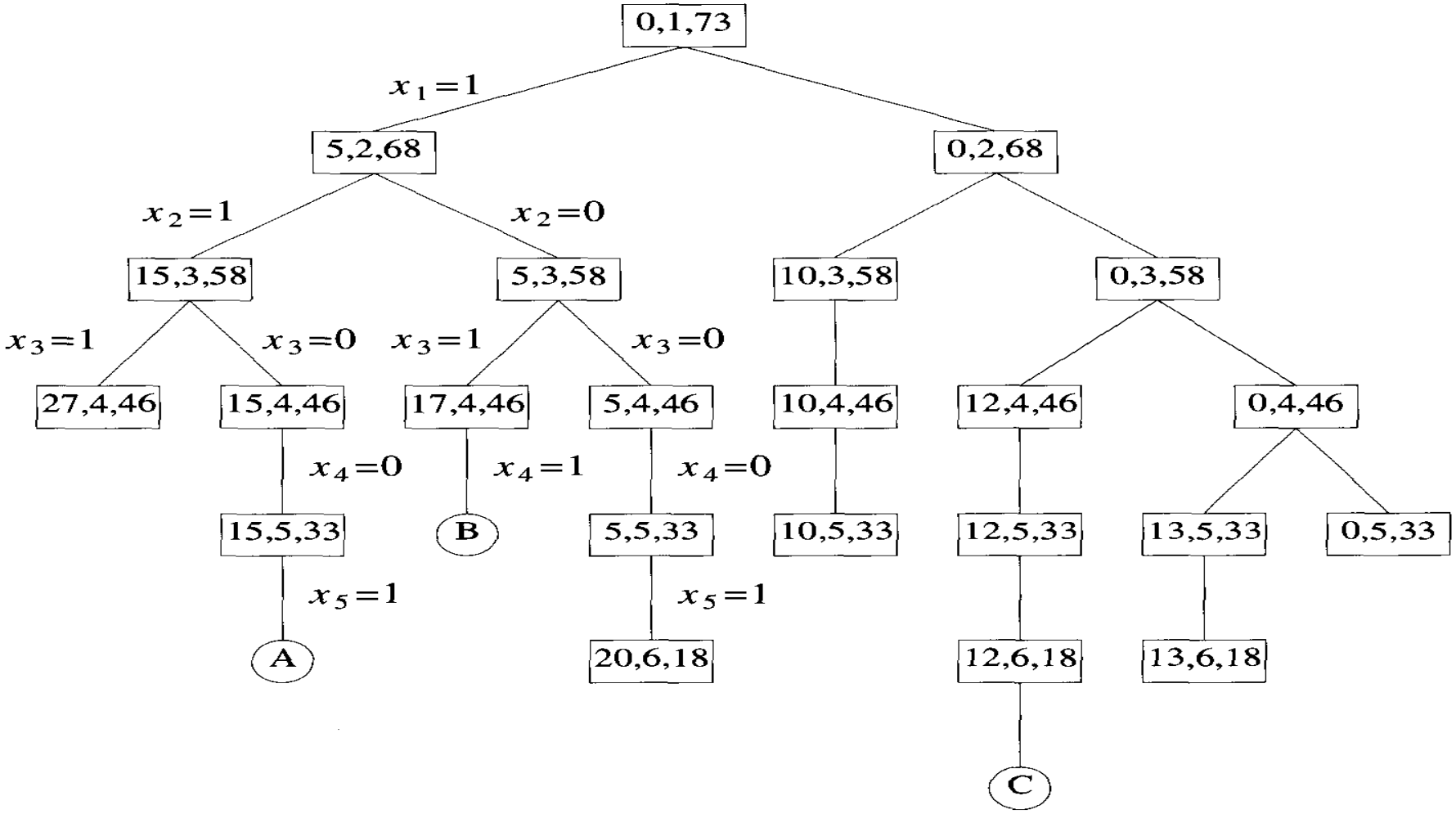
```
1   Algorithm SumOfSub(s, k, r)
2   // Find all subsets of w[1 : n] that sum to m. The values of x[j],
3   // 1 ≤ j < k, have already been determined. s = Σ_{j=1}^{k-1} w[j] * x[j]
4   // and r = Σ_{j=k}^{n} w[j]. The w[j]'s are in nondecreasing order.
5   // It is assumed that w[1] ≤ m and Σ_{i=1}^{n} w[i] ≥ m.
6   {
7       // Generate left child. Note: s + w[k] ≤ m since B_{k-1} is true.
8       x[k] := 1;
9       if (s + w[k] = m) then write (x[1 : k]); // Subset found
10          // There is no recursive call here as w[j] > 0, 1 ≤ j ≤ n.
11      else if (s + w[k] + w[k + 1] ≤ m)
12          then SumOfSub(s + w[k], k + 1, r − w[k]);
13      // Generate right child and evaluate B_k.
14      if ((s + r − w[k] ≥ m) and (s + w[k + 1] ≤ m)) then
15      {
16          x[k] := 0;
17          SumOfSub(s, k + 1, r − w[k]);
18      }
19  }
```

Time Complexity is $O(2^n)$

ADITYA UNIVERSITY

Ex - 1:

Let n=6, m=30, and w[1:6] = { 5,10,12,13,15,18 }



ADITYA UNIVERSITY

Ex - 2:

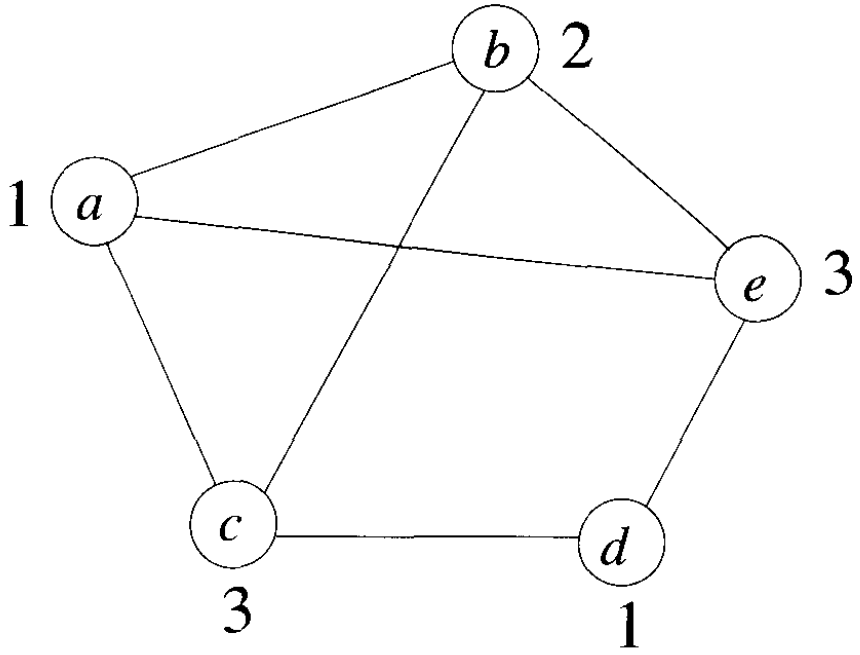
Let n=4, m=31, and w[1:4] = { 7, 11, 13, 24 }

ADITYA UNIVERSITY

3. Graph Coloring

A **coloring of a graph** is an assignment of colors to the vertices of the graph so that no two adjacent vertices have the same color.

- M-Color decision
- M-Color optimization
- Chromatic number
- Planner Graph
- Adjacency Matrix



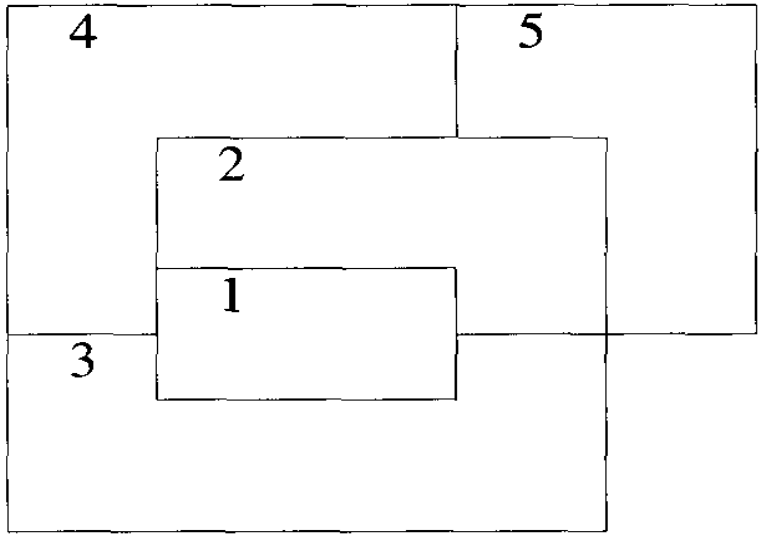
Key Terms

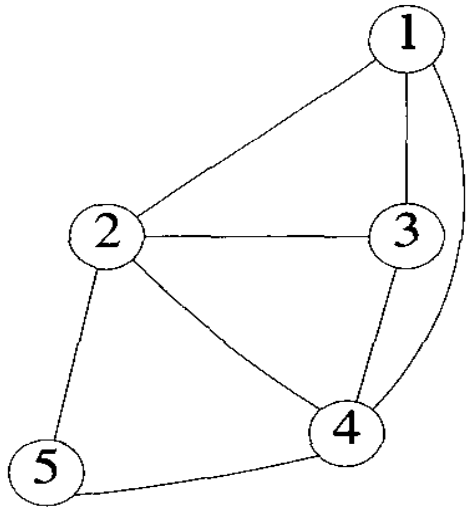
Given an undirected graph and a number m, determine if the graph can be colored with at most ***m colors*** such that no two adjacent vertices of the graph are colored with the same color.

The **chromatic number** of a graph is the smallest **number** of colors needed to color the vertices of so that no two adjacent vertices share the same color.

A ***planar graph*** is a graph that can be embedded in the plane, i.e., it can be drawn on the plane in such a way that its edges intersect only at their endpoints. In other words, it can be drawn in such a way that no edges cross each other.

Map and Planner Graph representation





	1	2	3	4	5
1	0	1	1	1	0
2	1	0	1	1	1
3	1	1	0	1	0
4	1	1	1	0	1
5	0	1	0	1	0

Algorithm

```
1   Algorithm mColoring(k)
2   // This algorithm was formed using the recursive backtracking
3   // schema. The graph is represented by its boolean adjacency
4   // matrix G[1 : n, 1 : n]. All assignments of 1, 2, . . . , m to the
5   // vertices of the graph such that adjacent vertices are
6   // assigned distinct integers are printed. k is the index
7   // of the next vertex to color.
8   {
9       repeat
10      {// Generate all legal assignments for x[k].
11          NextValue(k); // Assign to x[k] a legal color.
12          if (x[k] = 0) then return; // No new color possible
13          if (k = n) then     // At most m colors have been
14                              // used to color the n vertices.
15              write (x[1 : n]);
16          else mColoring(k + 1);
17      } until (false);
18  }
```

Contd ...

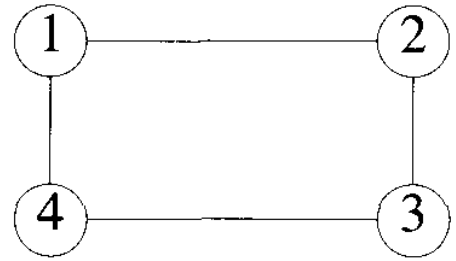
```
1    Algorithm NextValue(k)
2    // x[1], ..., x[k − 1] have been assigned integer values in
3    // the range [1, m] such that adjacent vertices have distinct
4    // integers. A value for x[k] is determined in the range
5    // [0, m]. x[k] is assigned the next highest numbered color
6    // while maintaining distinctness from the adjacent vertices
7    // of vertex k. If no such color exists, then x[k] is 0.
8    {
9        repeat
10       {
11           x[k] := (x[k] + 1) mod (m + 1); // Next highest color.
12           if (x[k] = 0) then return; // All colors have been used.
13           for j := 1 to n do
14           {    // Check if this color is
15                // distinct from adjacent colors.
16                if ((G[k, j] ≠ 0) and (x[k] = x[j]))
17                // If (k, j) is and edge and if adj.
18                // vertices have the same color.
19                     then break;
20           }
21           if (j = n + 1) then return; // New color found
22       } until (false); // Otherwise try to find another color.
23   }
```

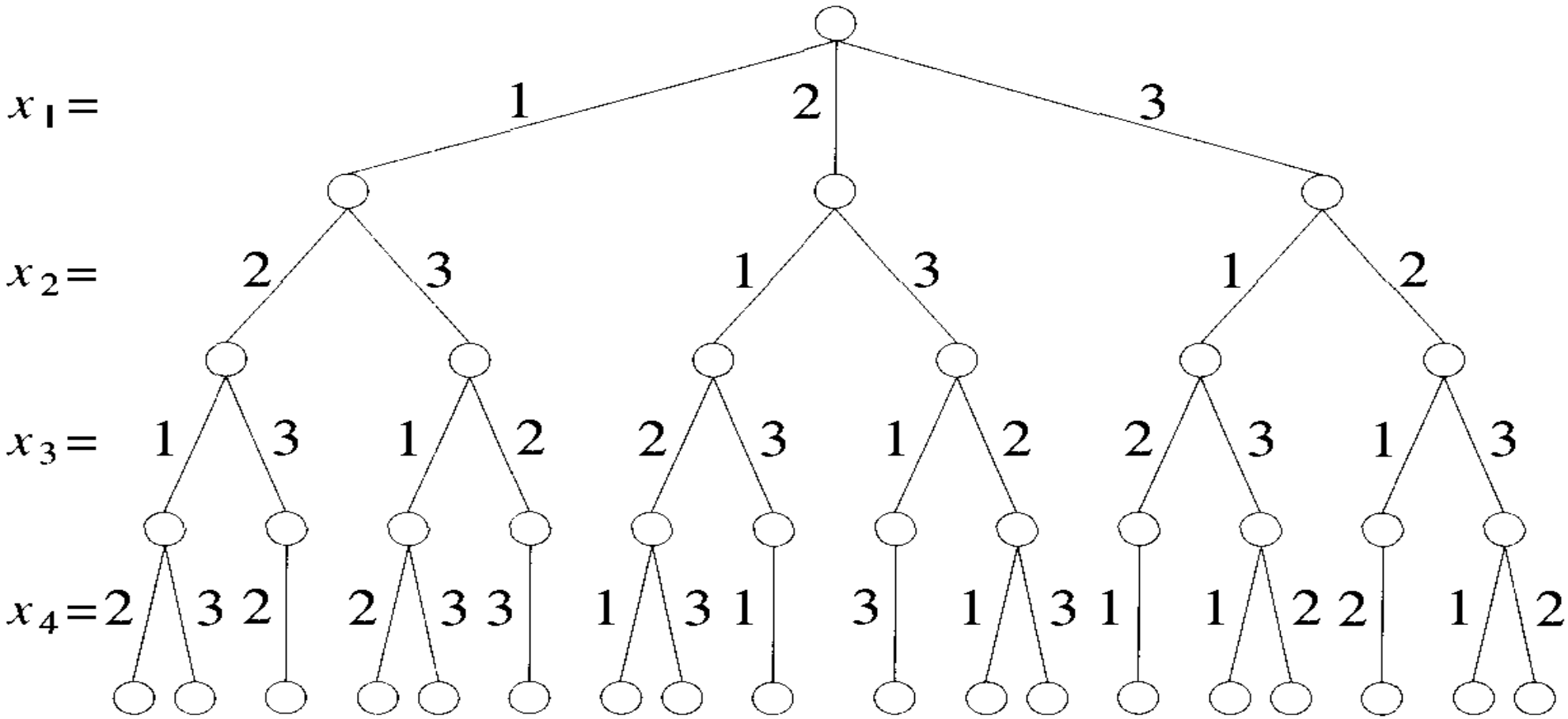
Time Complexity is $O(m^n)$

ADITYA UNIVERSITY

Ex:

n=4, m=3





ADITYA UNIVERSITY

4. Knapsack Problem

```
1   Algorithm BKnap(k, cp, cw)
2   // m is the size of the knapsack; n is the number of weights
3   // and profits. w[ ] and p[ ] are the weights and profits.
4   // p[i]/w[i] ≥ p[i + 1]/w[i + 1]. fw is the final weight of
5   // knapsack; fp is the final maximum profit. x[k] = 0 if w[k]
6   // is not in the knapsack; else x[k] = 1.
7   {
8       // Generate left child.
9       if (cw + w[k] ≤ m) then
10      {
11          y[k] := 1;
12          if (k < n) then BKnap(k + 1, cp + p[k], cw + w[k]);
13          if ((cp + p[k] > fp) and (k = n)) then
14          {
15              fp := cp + p[k]; fw := cw + w[k];
16              for j := 1 to k do x[j] := y[j];
17          }
18      }
19      // Generate right child.
20      if (Bound(cp, cw, k) ≥ fp) then
21      {
22          y[k] := 0; if (k < n) then BKnap(k + 1, cp, cw);
23          if ((cp > fp) and (k = n)) then
24          {
25              fp := cp; fw := cw;
26              for j := 1 to k do x[j] := y[j];
27          }
28      }
29  }
```

```
1   Algorithm Bound(cp, cw, k)
2   // cp is the current profit total, cw is the current
3   // weight total; k is the index of the last removed
4   // item; and m is the knapsack size.
5   {
6       b := cp; c := cw;
7       for i := k + 1 to n do
8       {
9           c := c + w[i];
9           if (c < m) then b := b + p[i];
10          else return b + (1 − (c − m)/w[i]) * p[i];
11      }
12      return b;
13  }
```

ADITYA UNIVERSITY

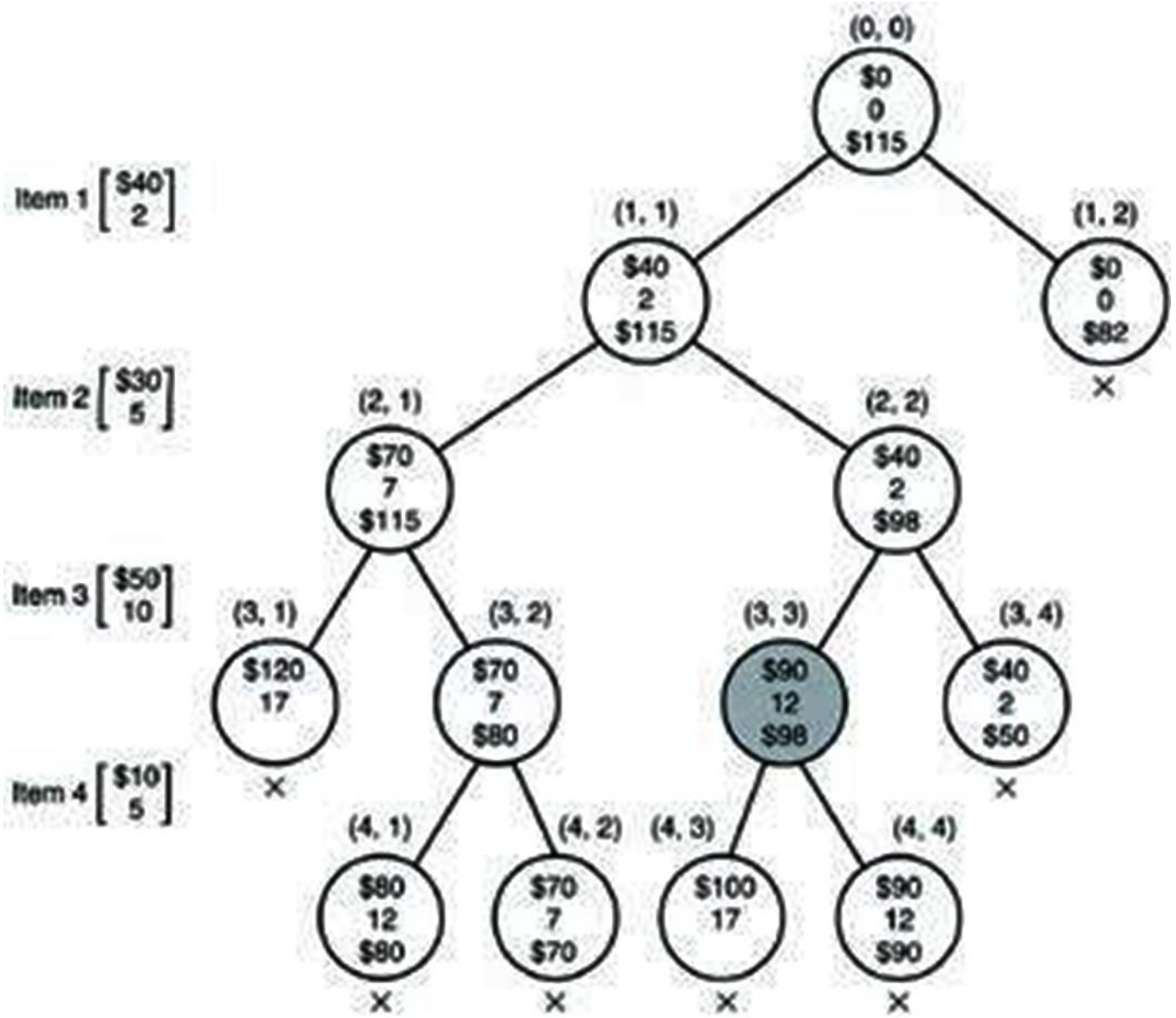
Example

Let n=4, M=16, and w[1:4] and v[1:4] are

V	40	30	50	10
W	2	5	10	5

ADITYA UNIVERSITY

Contd ...



5. Branch and Bound

GENERAL METHODE

Branch and Bound refers to all state space search methods in which all children of the E-Node are generated before any other live node becomes the E-Node.

- DFS, BFS, hill climbing, and best-first search can be used to solve some searching problem for searching a feasible solution.

- However, they cannot be used to solve the optimization problems for searching an (the) optimal solution.

ADITYA UNIVERSITY

Branch-and-bound strategy

- It is efficient in the average case because many branches can be terminated very early.
- Although it is usually very efficient, a very large tree may be generated in the worst case.
- Many NP-hard problem can be solved by B&B efficiently in the average case; however, the worst-case time complexity is still exponential.

Contd ...

- **Generally, we have THREE types of search strategies:**

- ***FIFO*** (First In First Out) – It is also called as BFS like state space search
- ***LIFO*** (Last In First Out) - It is also called as D-search like state space search
- ***LC*** (Least Cost) Search

ADITYA UNIVERSITY

Example

Let’s through an example.

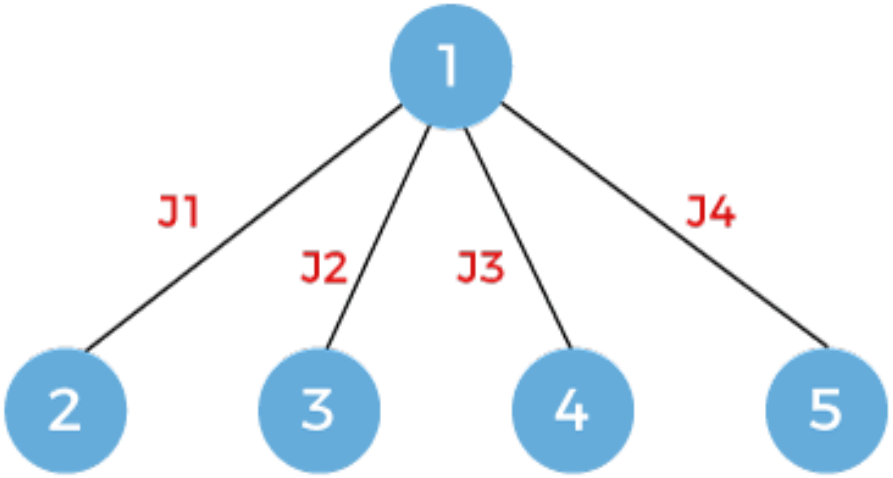
Jobs = {j1, j2, j3, j4}

P = {10, 5, 8, 3}

d = {1, 2, 1, 2}

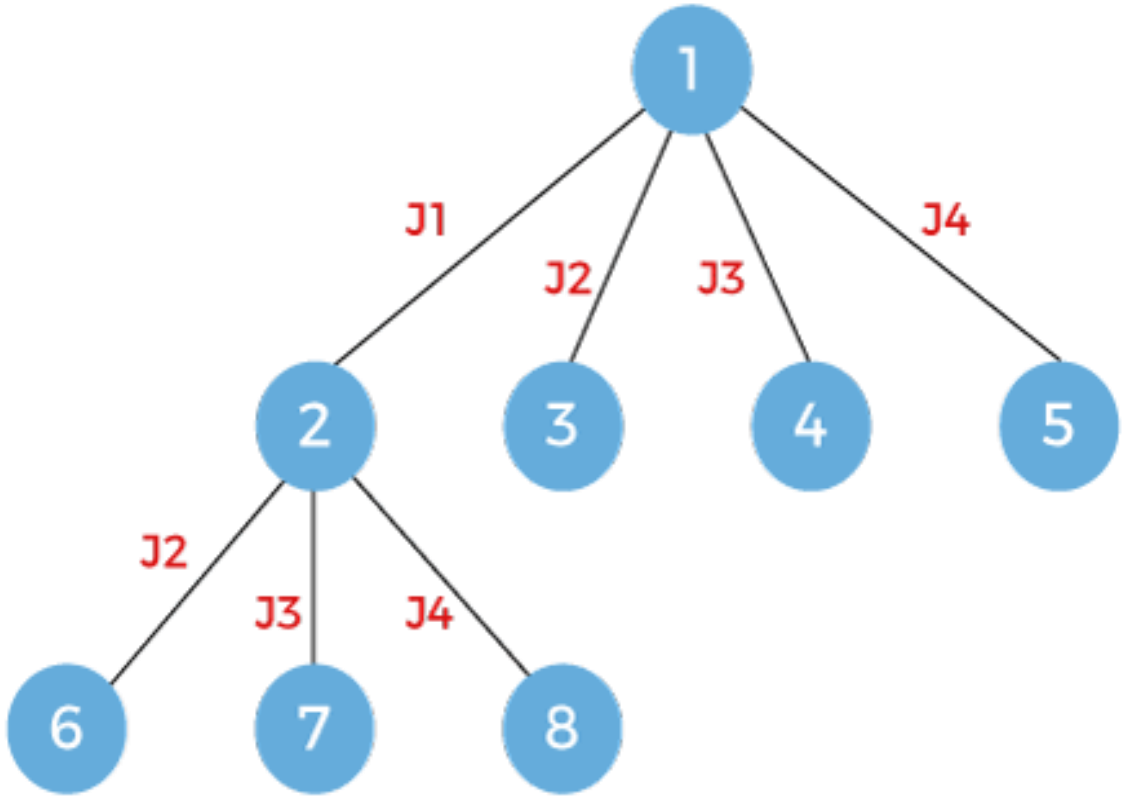
FIFO B&B

- In this case, we first consider the first job, then second job, then third job and finally we consider the last job.



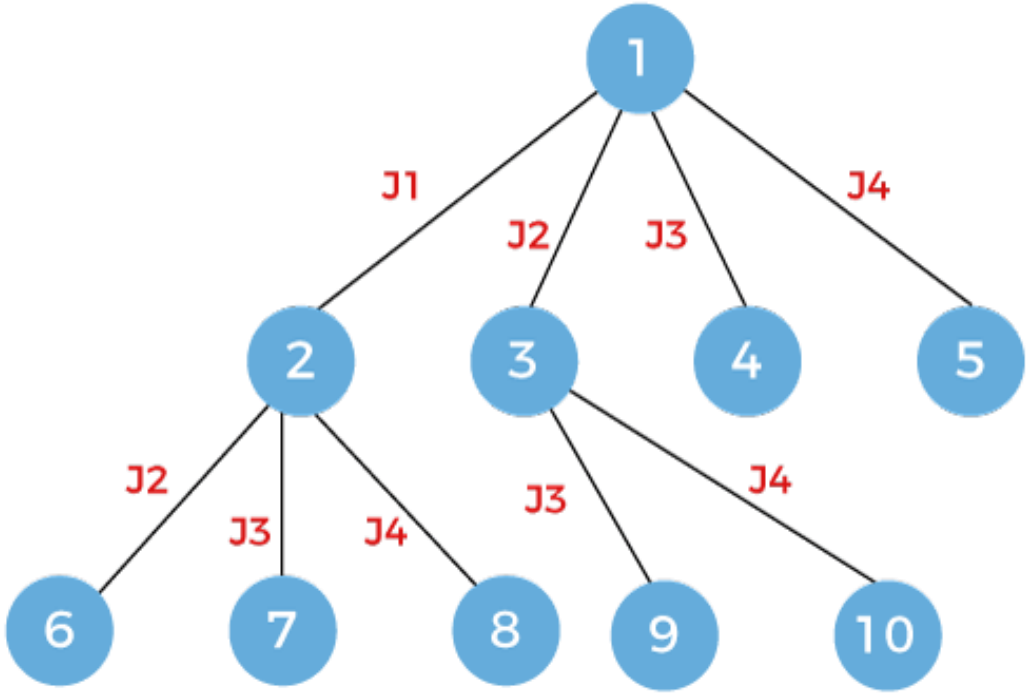
Contd ...

- Now one level is completed.
- Once take first job J1.



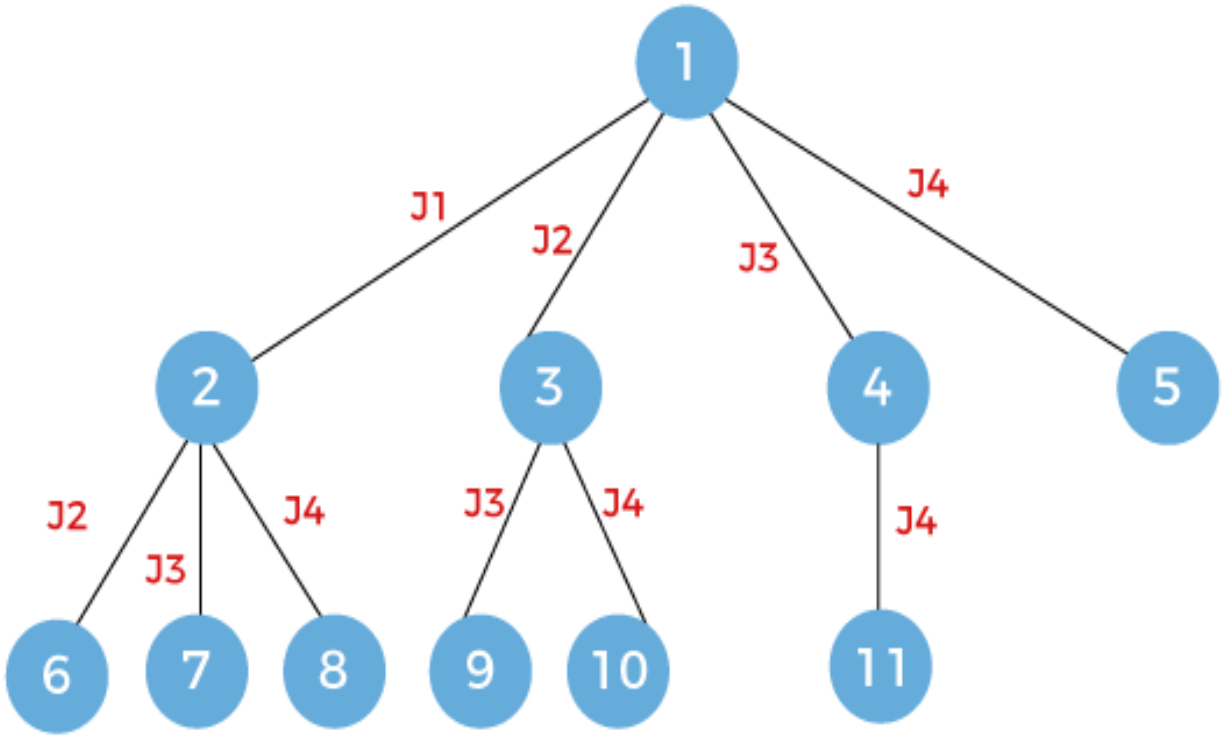
Contd ...

- Now we will consider the node 3.



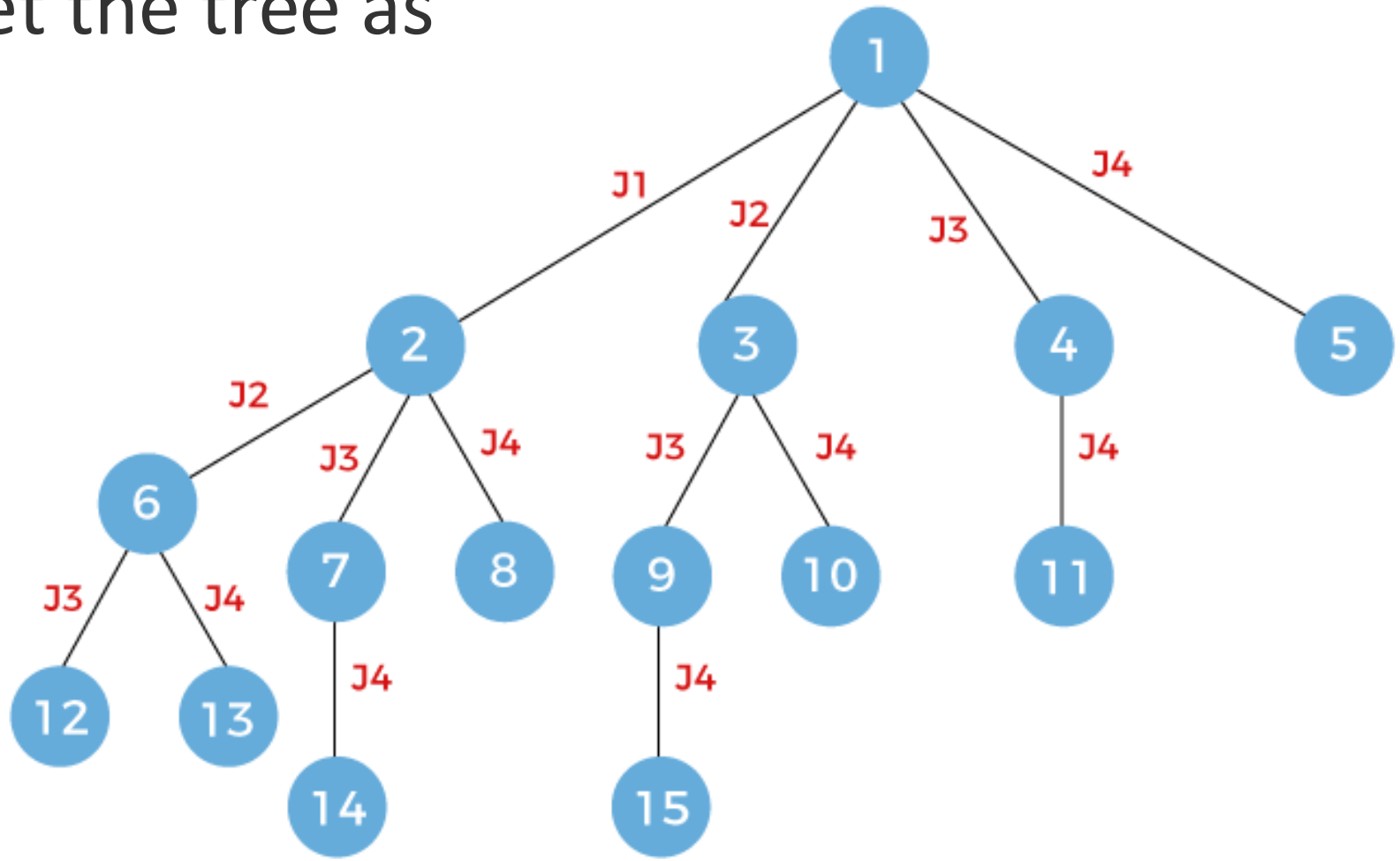
Contd ...

- Now we will expand the node 4. Since here we are doing job j3 so we will consider only job j4.



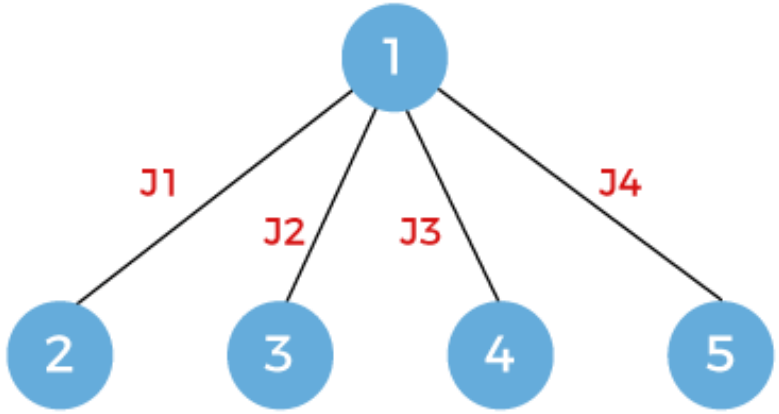
Contd ...

- Finally, we get the tree as



LIFO B&B

- First, we consider the node 1 shown as below:
- Now, we will expand the node 1. After expansion, the state space tree would be appeared as:
- On each expansion, the node will be pushed into the stack shown as below:





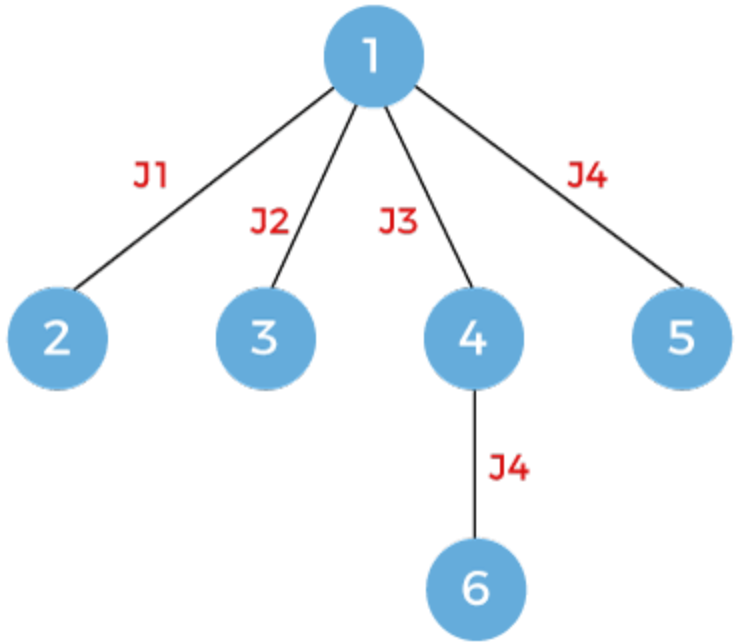
Contd ...

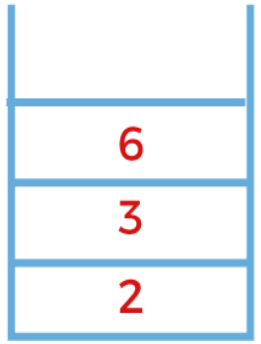
- Now the expansion would be based on the node that appears on the top of the stack. Since the node 5 appears on the top of the stack, so we will expand the node 5. We will pop out the node 5 from the stack. Since the node 5 is in the last job, i.e., j4 so there is no further scope of expansion.



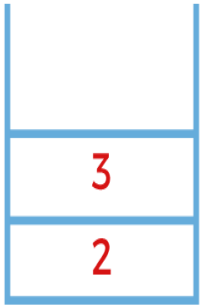
- The next node that appears on the top of the stack is node 4. Pop out the node 4 and expand. On expansion, job j4 will be considered and node 6 will be added into the stack shown as below:

Contd ...



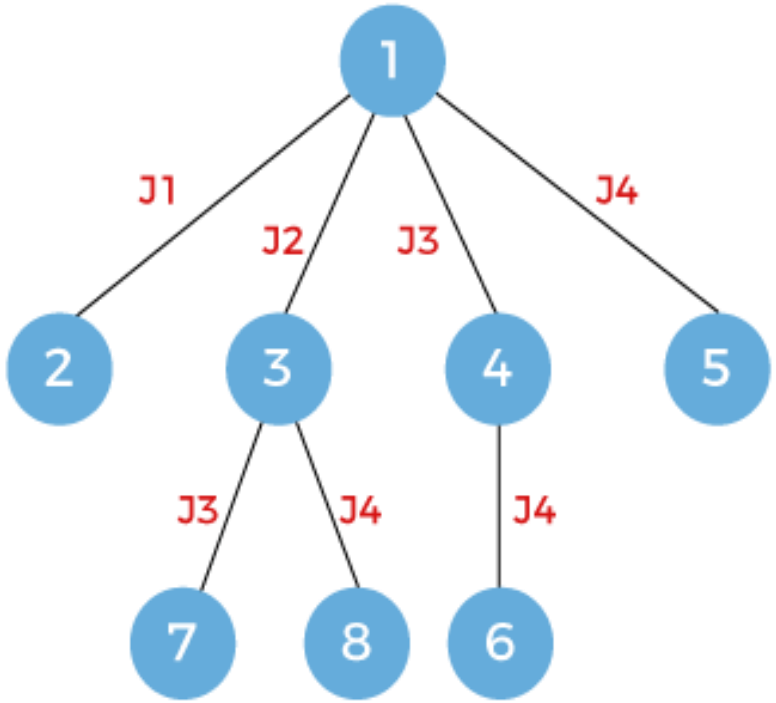


- The next node is 6 which is to be expanded. Pop out the node 6 and expand. Since the node 6 is in the last job, i.e., j4 so there is no further scope of expansion.



Contd ...

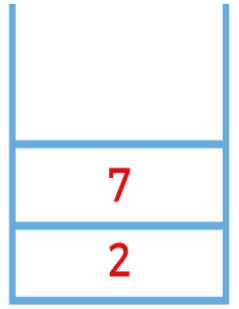
- The next node to be expanded is node 3. Since the node 3 works on the job j2 so node 3 will be expanded to two nodes, i.e., 7 and 8 working on jobs 3 and 4 respectively. The nodes 7 and 8 will be pushed into the stack shown as below:



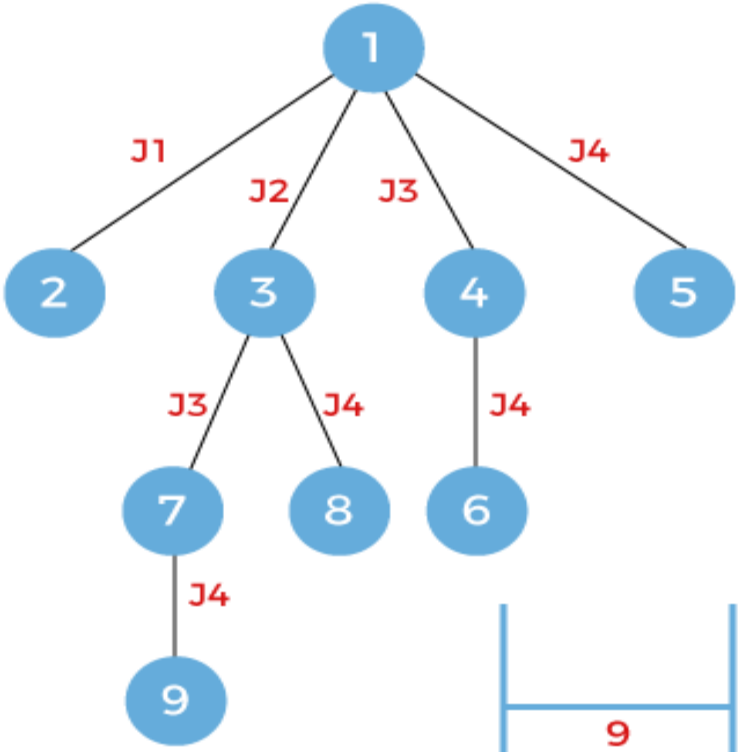


Contd ...

- The next node that appears on the top of the stack is node 8. Pop out the node 8 and expand. Since the node 8 works on the job j4 so there is no further scope for the expansion.

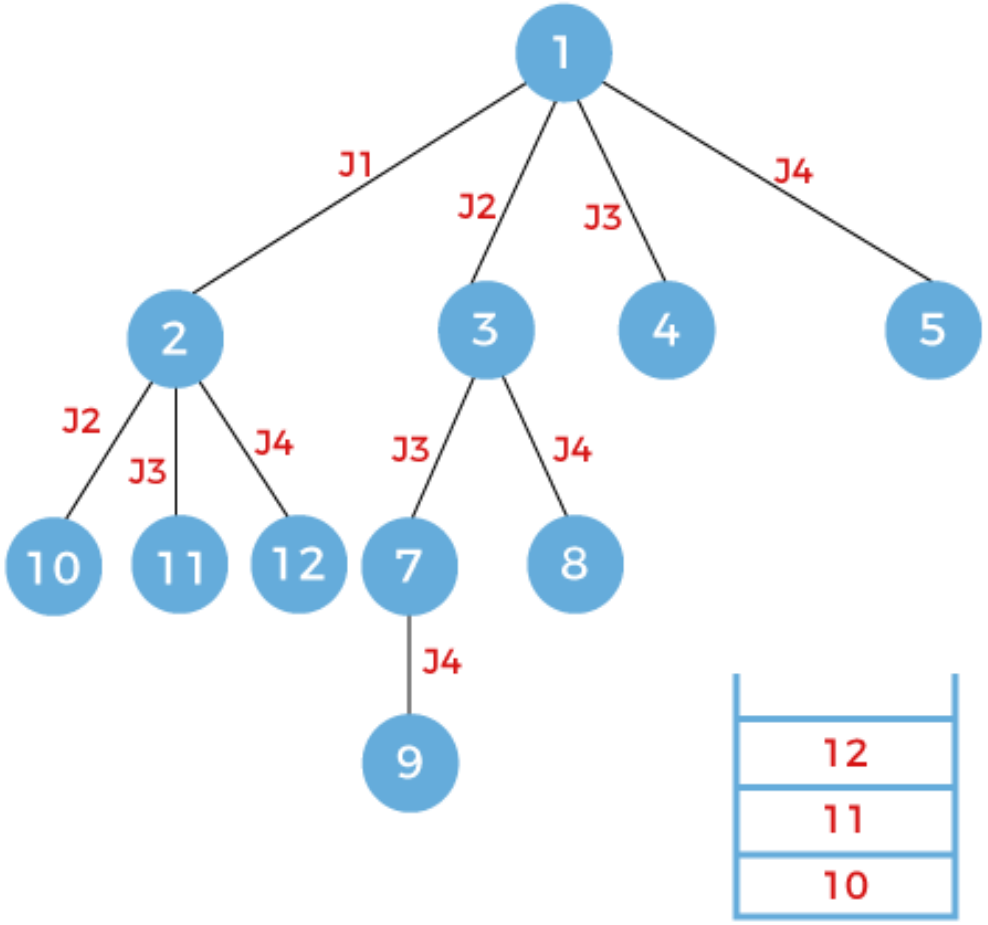


- The next node that appears on the top of the stack is node 7. Pop out the node 7 and expand. Since the node 7 works on the job j3 so node 7 will be further expanded to node 9 that works on the job j4 as shown as below and the node 9 will be pushed into the stack.



Contd ...

- If we apply this process, Finally we got this



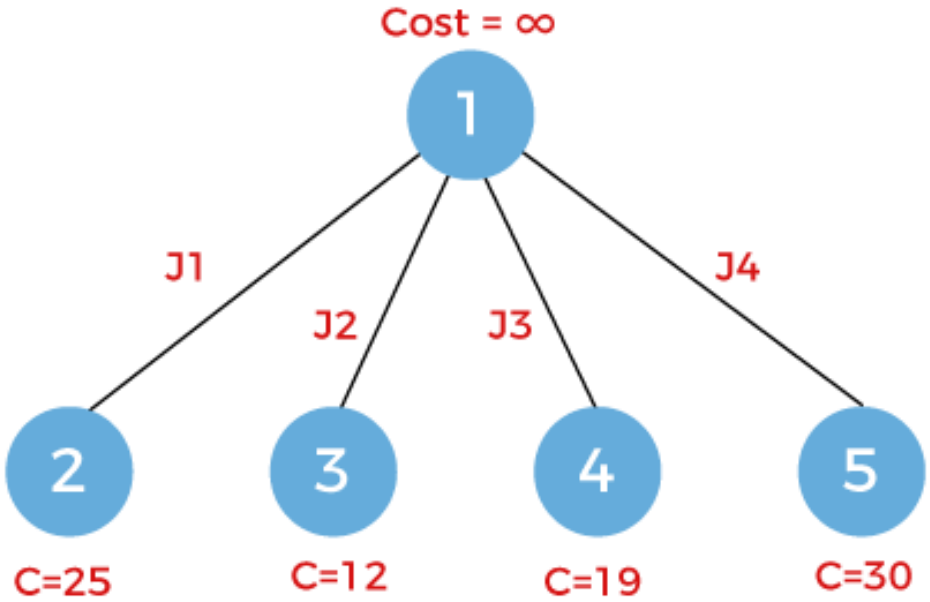
LC B&B

- There is one more method that can be used to find the solution and that method is Least cost branch and bound. In this technique, nodes are explored based on the cost of the node. The cost of the node can be defined using the problem and with the help of the given problem, we can define the cost function. Once the cost function is defined, we can define the cost of the node.

Contd ...

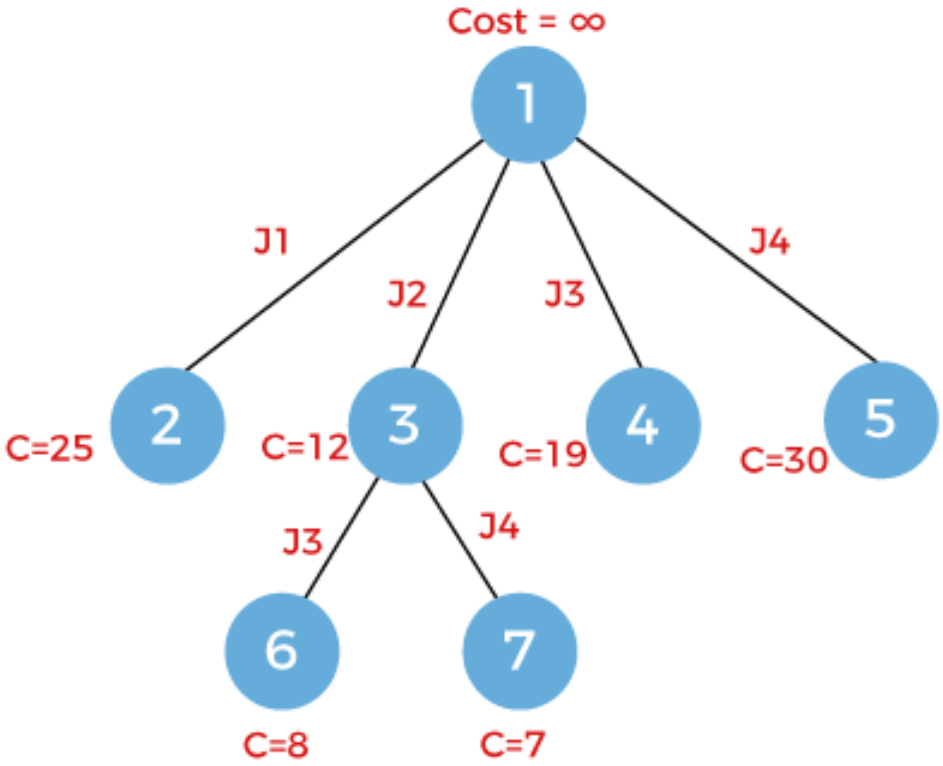
- Let's first consider the node 1 having cost infinity shown as below:

- Now we will expand the node 1. The node 1 will be expanded into four nodes named as 2, 3, 4 and 5 shown as below:



Contd ...

- Let's assume that cost of the nodes 2, 3, 4, and 5 are 25, 12, 19 and 30 respectively.
- Since it is the least cost branch n bound, so we will explore the node which is having the least cost. In the above figure, we can observe that the node with a minimum cost is node 3. So, we will explore the node 3 having cost 12.
- Since the node 3 works on the job j2 so it will be expanded into two nodes named as 6 and 7 shown as below:



6. 0/1 KNAPSACK
(by using Lower Bound and Upper Bound)

Example: Consider the instance M=15, n=4, $(p_1, p_2, p_3, p_4) = (10, 10, 12, 18)$ and $(w_1, w_2, w_3, w_4) = (2, 4, 6, 9)$.

First find the Bound Values at each and every node by using:

Upper Bound (U) : No fractional part is allowed to find it.

Lower Bound (L) : Fractional part is allowed to find it.

After that find the U-L, at which node we get min. consider that node.

Initially at Node (1)

Upper bound (U) =10+10+12=32

Lower bound (L) =10+10+12+ (3/9*18)=32+6=38

Note: Knapsack is maximization problem but branch bound technique is applicable for only minimization problems. In order to convert maximization problem into minimization problem we have to take negative sign for upper bound and lower bound i.e. U = -32 and L = -38.

Contd ...

At Node (2)

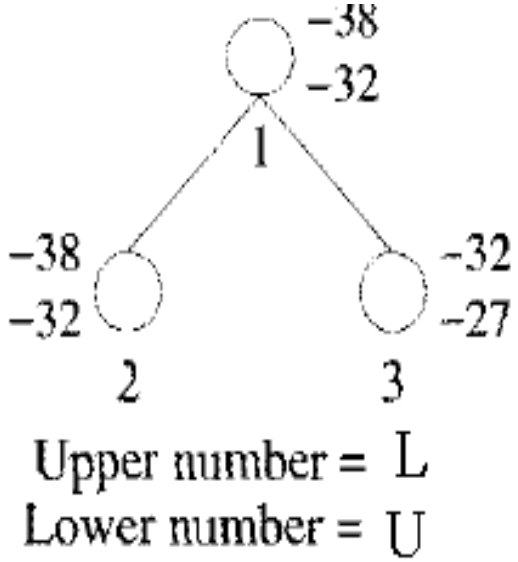
Upper bound (L) =10+10+12=32 make it as -32

Lower bound (U) =10+10+12+ (3/9*18)=32+6=38 make it as -38

At Node (3)

U=10+12=22, make it as -22

L=10+12+ (5/9*18) = 32, we make it as -32



Next we will calculate difference of upper bound and lower bound for nodes 2, 3

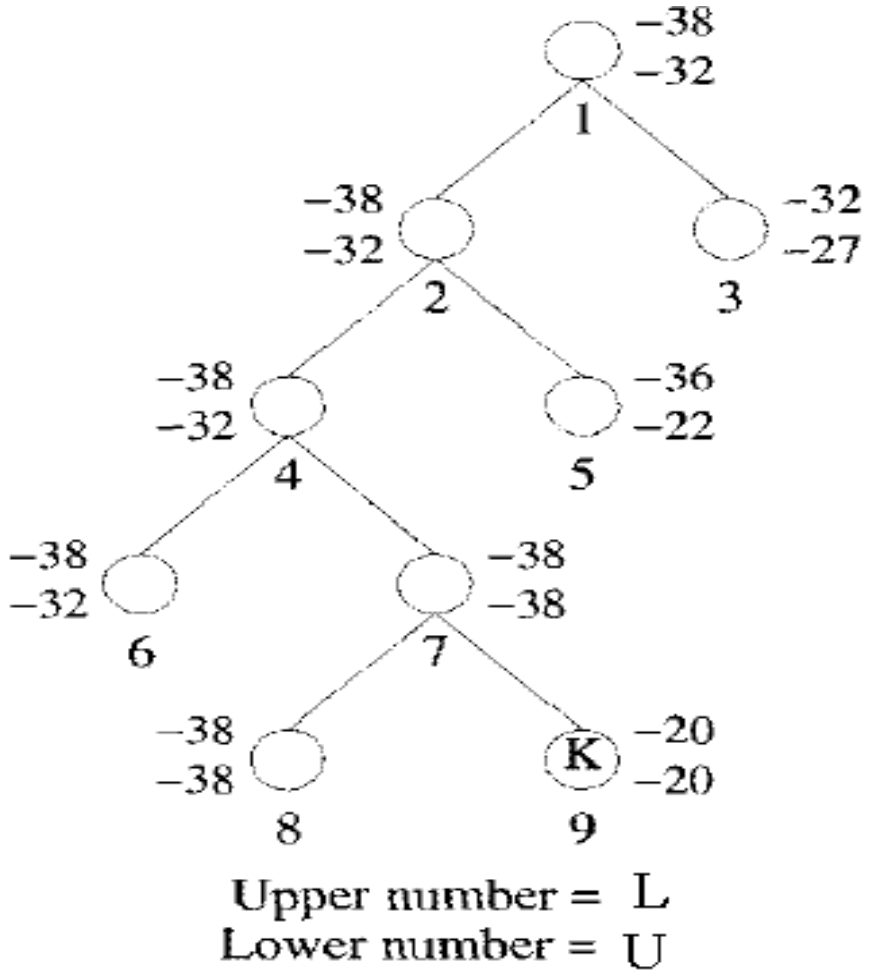
For node 2, U-L=-32+38=6

For node 3, U-L=-22+32=10

Choose node 2, since it has minimum difference value of 6.

Contd ...

We finding for all the nodes the tree as



For node 8, U-L=-38+38=0.
For node 9, U-L=-20+20=0.

Here, the difference is same, so we compare upper bounds of nodes 8 and 9. Discard the node, which has maximum upper bound

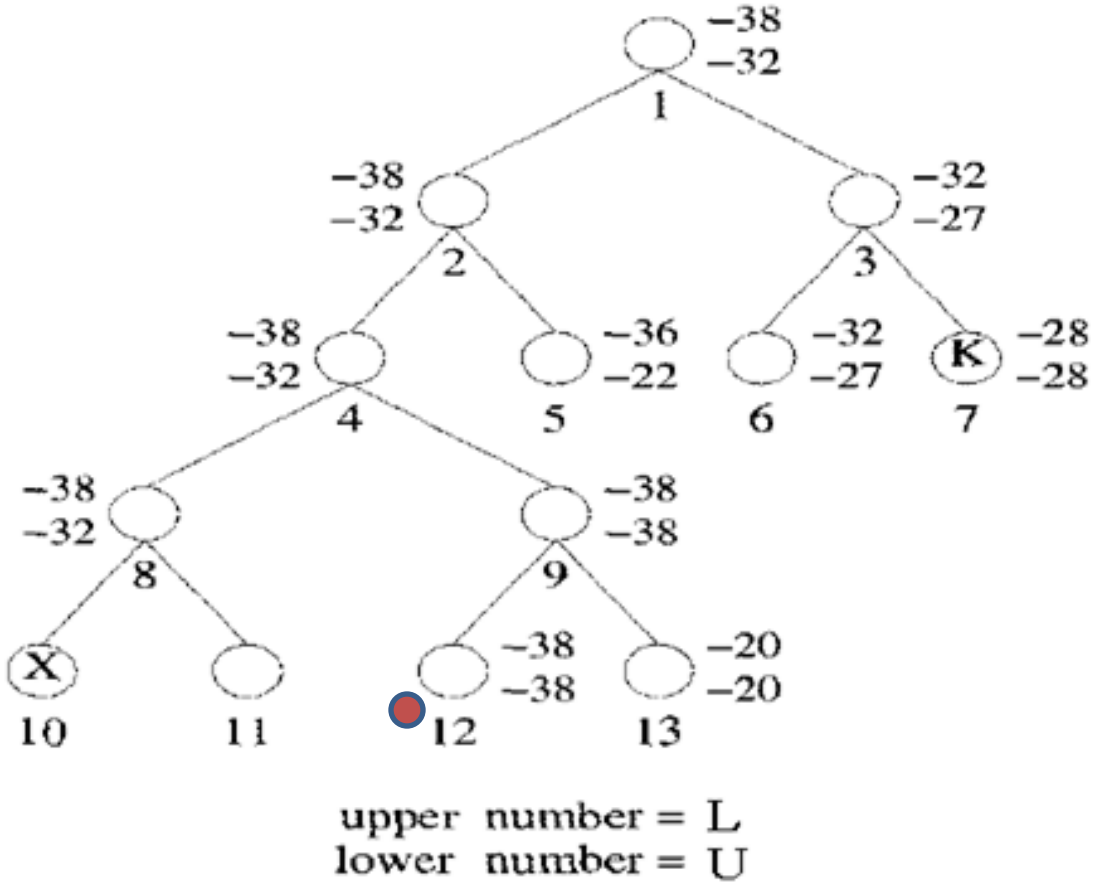
The Path is: **1→2→4→7→8**

The solution is (x_1, x_2, x_3,x_4) = (1, 1, 0, 1)

Maximum profit is:

$\sum p_i x_i = 10*1+10*1+12*0+18*1 = 10+10+18=38.$

0/1 KNAPSACK (FIFO)



X_1=1, X2=1, X3=0, X4=1

The traveling salesperson optimization problem

- Given a graph, the TSP Optimization problem is to find a tour, starting from any vertex, visiting every other vertex and returning to the starting vertex, with minimal cost.
- It is NP-complete
- We try to avoid n! exhaustive search by the branch-and-bound technique.

Contd ...

- E.g. A Cost Matrix for a Traveling Salesperson Problem.

i \ j	1	2	3	4	5	6	7
1	∞	3	93	13	33	9	57
2	4	∞	77	42	21	16	34
3	45	17	∞	36	16	28	25
4	39	90	80	∞	56	7	91
5	28	46	88	33	∞	25	57
6	3	88	18	46	92	∞	7
7	44	26	33	27	84	39	∞

ADITYA UNIVERSITY

The basic idea

- There is a way to split the solution space (branch)
- There is a way to predict a lower bound for a class of solutions. There is also a way to find a upper bound of an optimal solution. If the lower bound of a solution exceeds the upper bound, this solution cannot be optimal and thus we should terminate the branching associated with this solution.

ADITYA UNIVERSITY

Splitting

- We split a solution into two groups:
  1. One group including a particular arc
  2. The other excluding the arc
- Each splitting incurs a lower bound and we shall traverse the searching tree with the “lower” lower bound.

Contd ...

- Reduced cost matrix:

i \ j	1	2	3	4	5	6	7	
1	∞	0	90	10	30	6	54	(-3)
2	0	∞	73	38	17	12	30	(-4)
3	29	1	∞	20	0	12	9	(-16)
4	32	83	73	∞	49	0	84	(-7)
5	3	21	63	8	∞	0	32	(-25)
6	0	85	15	43	89	∞	4	(-3)
7	18	0	7	1	58	13	∞	(-26)
								reduced:84

A Reduced Cost Matrix.

Contd ...

i \ j	1	2	3	4	5	6	7
1	∞	0	83	9	30	6	50
2	0	∞	66	37	17	12	26
3	29	1	∞	19	0	12	5
4	32	83	66	∞	49	0	80
5	3	21	56	7	∞	0	28
6	0	85	8	42	89	∞	0
7	18	0	0	0	58	13	∞
			(-7)	(-1)			(-4)

Table: Another Reduced Cost Matrix.

ADITYA UNIVERSITY

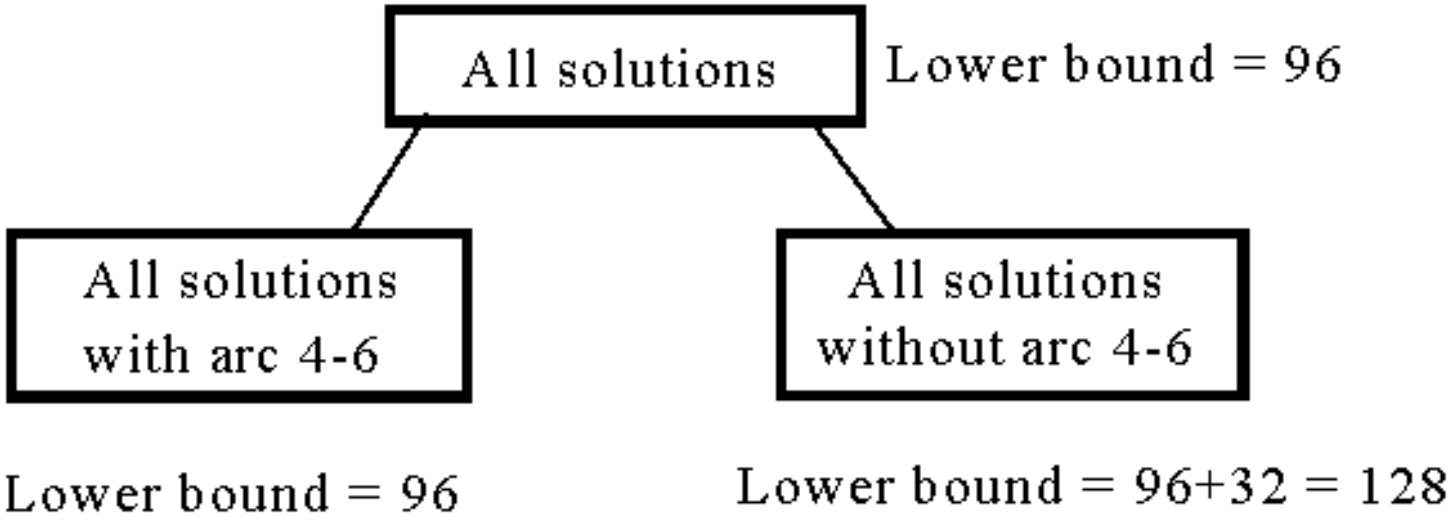
Lower bound

- The total cost of 84+12=96 is subtracted. Thus, we know the lower bound of this TSP problem is 96.

Contd ...

- Total cost reduced: 84+7+1+4 = 96 (lower bound)

decision tree:



- The Highest Level of a Decision Tree.
- If we use arc 3-5 to split, the difference on the lower bounds is 17+1 = 18.

ADITYA UNIVERSITY

For the right subtree

i \ j	1	2	3	4	5	7
1	∞	0	83	9	30	50
2	0	∞	66	37	17	26
3	29	1	∞	19	0	5
5	3	21	56	7	∞	28
6	0	85	8	∞	89	0
7	18	0	0	0	58	∞

A Reduced Cost Matrix if Arc 4-6 is Included.
(We must set c6-4 to be ∞)

For the left subtree

- The cost matrix for all solution with arc 4-6:

i \ j	1	2	3	4	5	7	
1	∞	0	83	9	30	50	
2	0	∞	66	37	17	26	
3	29	1	∞	19	0	5	
5	0	18	53	4	∞	25	(-3)
6	0	85	8	∞	89	0	
7	18	0	0	0	58	∞	

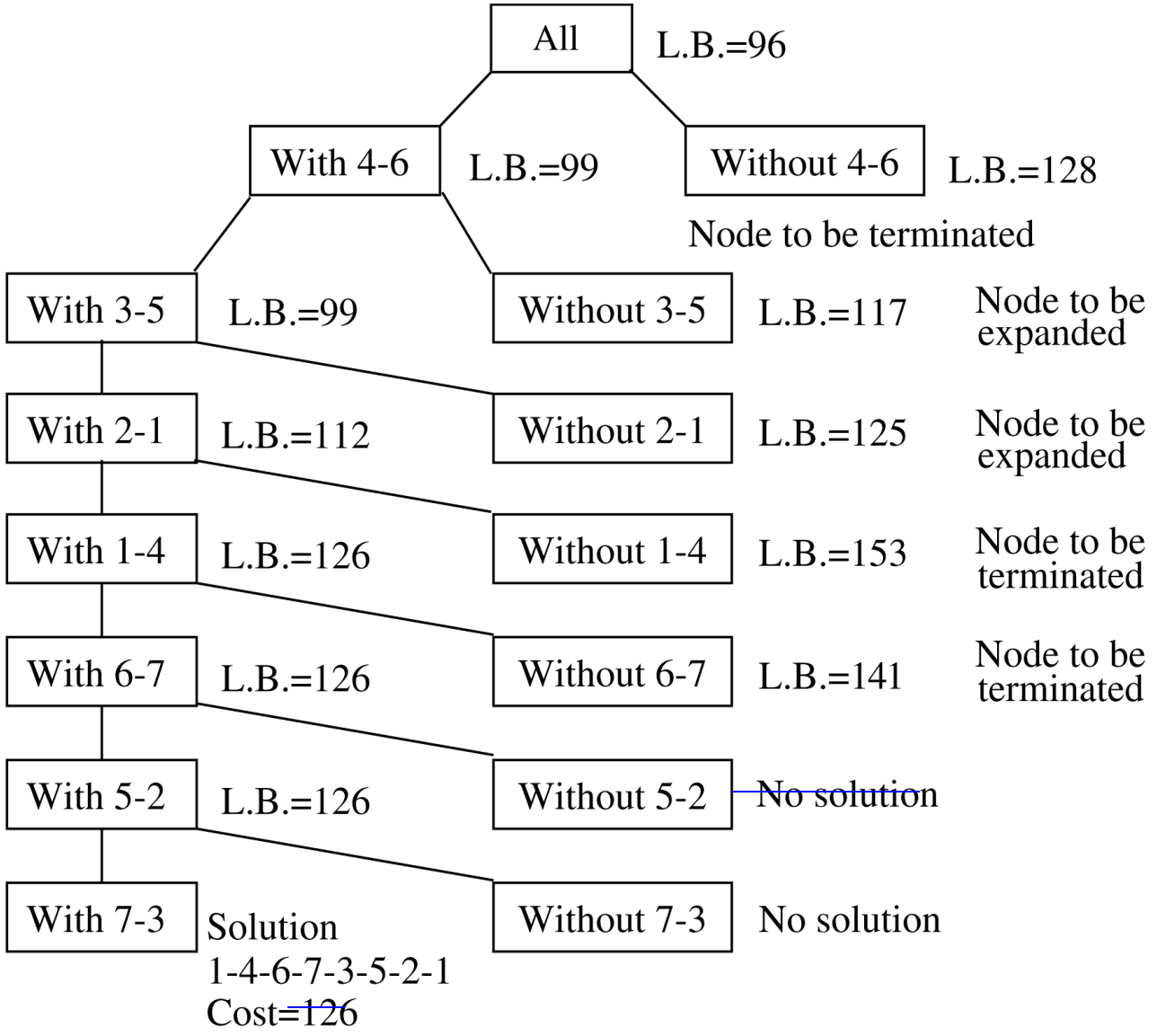
- Total cost reduced: 96+3 = 99 (new lower bound)

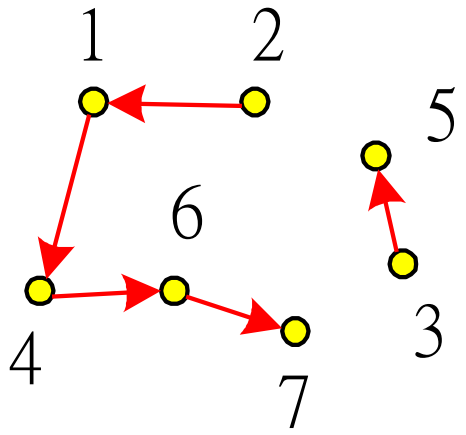
63

ADITYA UNIVERSITY

Upper bound

- We follow the best-first search scheme and can obtain a feasible solution with cost C.
- C serves as an upper bound and many branching's may be terminated if their lower bounds exceed C.


All
L.B.=96
With 4-6
L.B.=99
Without 4-6
L.B.=128
Node to be terminated
With 3-5
L.B.=99
Without 3-5
L.B.=117
Node to be expanded
With 2-1
L.B.=112
Without 2-1
L.B.=125
Node to be expanded
With 1-4
L.B.=126
Without 1-4
L.B.=153
Node to be terminated
With 6-7
L.B.=126
Without 6-7
L.B.=141
Node to be terminated
With 5-2
L.B.=126
Without 5-2
No solution
With 7-3
Solution
1-4-6-7-3-5-2-1
Cost=126
Without 7-3
No solution


1
2
5
6
4
7
3

ADITYA UNIVERSITY

Preventing an arc

- In general, if paths i_1-i_2-…-i_m and j_1-j_2-…-j_n have already been included and a path from i_m to j_1 is to be added, then path from j_n to i_1 must be prevented (by assigning the cost of j_n to i_1 to be ∞)